Etude de cas en classification et prédiction.

Analyse du jeu de données.

1- Importer le jeu de données EANN fourni sur moodle. Vérifiez que vous avez bien 700 observations avec plusieurs paramètres enregistrés pour chaque observation.

In [1]:
```
import warnings
warnings.simplefilter(action='ignore', category=FutureWarning)
import pandas as pd
import matplotlib.pyplot as plt
import seaborn

# Importer le jeu de donnée.
# On peut voir en ouvrant le fichier que le separateur est ";"
df = pd.read_csv("BaseEANN100x7.csv", sep=";")

# vérifier l'importation
print("Ce dataframe contient",df.shape[0],"lignes et",df.shape[1],"colones.")
print(df.head(5))
```

```
Ce dataframe contient 700 lignes et 33 colones.
     lfws   ifws  hfws  nbfws  lsws_ls  isws_ls  hsws_ls  nbsws_ls  lfly_ls  \
0  115.67  90749  1554      4   123.74     8430      199         4   122.85
1  120.16  24678   550      6   125.54     2463       54         3   125.54
2  104.02  27149   374      6   104.02     1839       31         4    90.57
3  105.81  28778   541      5   111.19     1305       19         3   106.71
4   75.32  24697   676      4    90.57     2346       64         3    56.49

   ifly_ls  ...  nbsws_hs  lfly_hs  ifly_hs  hfly_hs  nbfly_hs  lflr_hs  \
0      218  ...         7   127.33     9530      149         5   120.16
1      151  ...         6   125.54     5060       94         4   124.64
2      101  ...         6   104.02     4240       80         6   100.43
3      131  ...         6   111.19     4373       68         5   108.50
4       85  ...         3    90.57     4227      108         4    78.01

    iflr_hs  hflr_hs  nbflr_hs                class
0  227707.0     3612         6  chaetoceros_socialis
1  109129.0     2064         6  chaetoceros_socialis
2   86116.0     1920         6  chaetoceros_socialis
3   90006.0     1754         6  chaetoceros_socialis
4   90646.0     2259         4  chaetoceros_socialis

[5 rows x 33 columns]
```

Validité des données

In [2]:
```
# bref aperçu des statistiques
stats=df.describe()
stats.loc['count'] = stats.loc['count'].astype(int).astype(str)
stats.iloc[1:] = stats.iloc[1:].applymap('{:.1f}'.format)
print(stats.T)
```

	count	mean	std	min	25%	50%	75%	\
lfws	700	100.6	122.9	3.0	18.1	72.2	123.7	
ifws	700	29704.7	38764.0	415.0	8396.8	17005.5	32587.5	
hfws	700	537.4	395.3	23.0	220.0	516.0	775.0	
nbfws	700	4.4	4.9	0.0	1.0	3.0	6.0	
lsws_ls	700	103.9	124.8	3.0	22.5	73.5	128.3	
isws_ls	700	6093.9	8560.3	79.0	660.2	3311.0	7322.2	
hsws_ls	700	98.5	108.2	8.0	32.0	55.5	132.5	
nbsws_ls	700	3.3	3.3	0.0	1.0	2.0	4.0	
lfly_ls	700	579.9	2164.1	0.2	23.4	68.0	122.0	
ifly_ls	700	124.9	144.3	0.0	24.0	90.5	151.0	
hfly_ls	700	1.7	1.1	0.0	1.0	1.0	2.0	
nbfly_ls	700	0.0	0.1	0.0	0.0	0.0	0.0	
lflo_ls	700	944.0	2800.9	0.2	26.0	65.9	123.7	
iflo_ls	700	93.6	107.5	0.0	18.0	63.0	119.2	
hflo_ls	700	1.2	0.6	0.0	1.0	1.0	2.0	
nbflo_ls	700	0.0	0.0	0.0	0.0	0.0	0.0	
lflr_ls	700	97.7	111.9	1.4	21.9	70.9	124.0	
iflr_ls	700	2759.0	4423.7	13.0	140.5	1669.5	3047.2	
hflr_ls	700	41.0	38.0	1.0	7.0	27.0	72.0	
nbflr_ls	700	2.9	3.9	0.0	1.0	2.0	4.0	
lsws_hs	700	103.5	125.1	3.0	19.0	74.4	128.3	
isws_hs	700	177382.2	222076.9	3562.0	22828.0	112165.5	208908.0	
hsws_hs	700	2449.1	1438.1	249.0	962.5	2364.0	4026.0	
nbsws_hs	700	4.8	6.0	1.0	1.0	3.0	6.0	
lfly_hs	700	103.7	123.7	2.7	23.4	73.5	127.3	
ifly_hs	700	5840.4	9284.5	58.0	322.5	3714.0	6304.8	
hfly_hs	700	83.1	78.9	4.0	14.0	49.0	149.0	
nbfly_hs	700	4.5	5.0	0.0	1.0	3.0	6.0	
lflr_hs	700	99.1	121.4	3.0	19.6	69.9	124.0	
iflr_hs	700	124852.9	205825.9	346.0	3766.0	72221.0	134491.5	
hflr_hs	700	1670.7	1593.0	30.0	179.5	975.0	3580.0	
nbflr_hs	700	4.2	4.8	0.0	1.0	2.0	6.0	

	max
lfws	923.8
ifws	285133.0
hfws	2558.0
nbfws	31.0
lsws_ls	926.4
isws_ls	82597.0
hsws_ls	1362.0
nbsws_ls	25.0
lfly_ls	9999.0
ifly_ls	858.0
hfly_ls	5.0
nbfly_ls	1.0
lflo_ls	9999.0
iflo_ls	609.0
hflo_ls	3.0
nbflo_ls	0.0
lflr_ls	759.6
iflr_ls	31805.0
hflr_ls	150.0
nbflr_ls	25.0
lsws_hs	926.4
isws_hs	1265890.0
hsws_hs	4327.0
nbsws_hs	50.0
lfly_hs	923.0
ifly_hs	65568.0
hfly_hs	382.0
nbfly_hs	29.0
lflr_hs	915.2
iflr_hs	1346450.0
hflr_hs	4137.0
nbflr_hs	32.0

Q2 Les données sont-elles toutes complètes ?

On peut observer que la ligne nbflo_ls ne contient que des 0, elle est donc inutile. On peut également voir que la ligne nbfly_ls a un ecart type trés proche de 0, et a donc peu d'intérêt, tout comme la ligne hflo_ls.

Q3 Quel est l'intérêt ici de la propriété T du dataframe stats ?

L'intérêt de la propriété T est de transposer l'affichage des statistiques pour avoir les statistiques par colonne et non par ligne, ce qui permet une lecture plus facile lorque l'on dispose d'un jeu de données avec un nombre important de colone.

Dans ce résumé statistique, nous voyons que des variables ont des écart-types nul. Ceci correspond a une colonne

n'ayant qu'une valeur. Elle n'a donc aucun intérêt dans une classification et devra être retirée.

In [3]:
```
# TO DO : Ecrire permettant de conserver les colonnes de valeurs non uniques
agarder = [] # à remplir
colsuppr = {} # contient la valeur moyenne des colonnes supprimée
epsilon = 0.1
for col in stats:
    if float(stats[col]['std']) >= epsilon:
        agarder.append(col)
    else:
        colsuppr[col] = float(stats[col]['mean'])

df_temp = df.copy()
for col in df_temp :
    if col not in agarder and col != "class":
        df.drop(columns=col, inplace=True)
        print("drop", col)
print('nouvelles dimensions= ',df.shape)
print(colsuppr)

# Correction
# agarder []
# agarder = [col for col in df.columns if len(df[col].unique()) > 1] # Renvoie les colones non-constantes
# print(agarder)
# df = df[agarder].copy()
# print('nouvelles dimensions = ', df.shape)
```

```
drop nbflo_ls
nouvelles dimensions=  (700, 32)
{'nbflo_ls': 0.0}
```

Pour ne garder que les colones qui ont un intérêt, j'ai défini une valeur en dessous de laquelle on considère que l'ecart type est suffisament proche de 0. Par precaution, j'ai gardé la valeur moyenne des colones supprimé dans un dictionnaire.

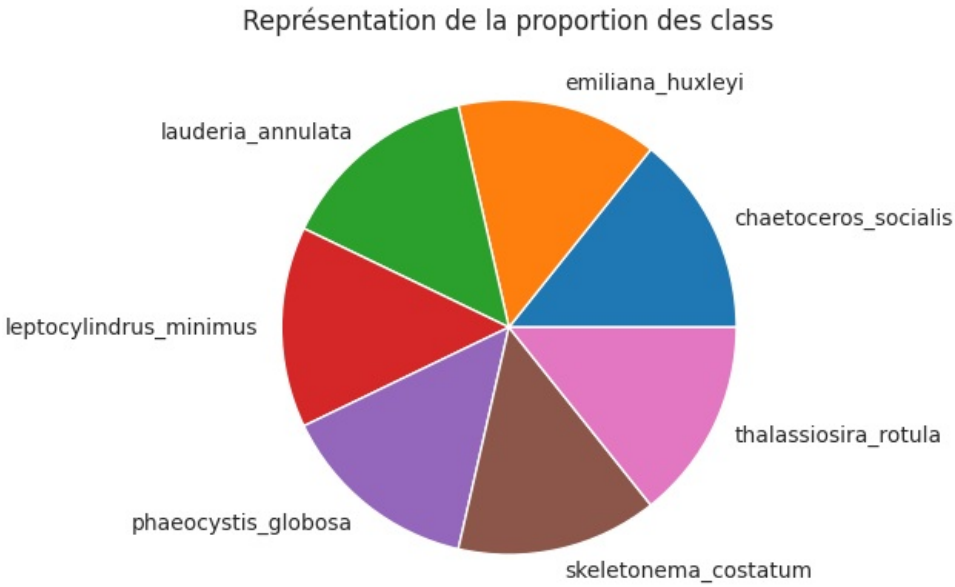
Q4 Répérer la colonne nommée class contenant les labels des observations, la sauvegarder au besoin dans une variable et la retirer du dataframe. Afficher la répartition des classes, est-ce un problème équilibré ? Vous remarquerez que les noms des classes sont des noms d'espèces phytoplanctoniques communes en Manche Mer du Nord.

In [4]:
```
# TO DO
print(df.columns)
labels=df["class"]
print('nom des especes= ',labels.unique())

# Affichage des proportions
comptage = df.groupby('class').size().to_dict()
total = sum(comptage.values())
proportion = {key: 100 *value / total for key, value in comptage.items()}
plt.pie(df['class'].value_counts(),labels=df['class'].value_counts().index)
plt.title("Représentation de la proportion des class")
plt.show()

# Suppression de la colone class
df.drop(columns="class", inplace=True)
```

```
Index(['lfws', 'ifws', 'hfws', 'nbfws', 'lsws_ls', 'isws_ls', 'hsws_ls',
       'nbsws_ls', 'lfly_ls', 'ifly_ls', 'hfly_ls', 'nbfly_ls', 'lflo_ls',
       'iflo_ls', 'hflo_ls', 'lflr_ls', 'iflr_ls', 'hflr_ls', 'nbflr_ls',
       'lsws_hs', 'isws_hs', 'hsws_hs', 'nbsws_hs', 'lfly_hs', 'ifly_hs',
       'hfly_hs', 'nbfly_hs', 'lflr_hs', 'iflr_hs', 'hflr_hs', 'nbflr_hs',
       'class'],
      dtype='object')
nom des especes=  ['chaetoceros_socialis' 'emiliana_huxleyi' 'lauderia_annulata'
 'leptocylindrus_minimus' 'phaeocystis_globosa' 'skeletonema_costatum'
 'thalassiosira_rotula']
```



On peut observer que les 7 classes ont une repartition identiques, le probléme est donc équilibré.

Etude des redondances

Le nombre de variables est important. La visualisation globale par paire n'est donc pas adéquate pour étudier les corrélations possibles.

Il est important de calculer la matrice de corrélation et faire une analyse des redondances élevées de manière automatique (par du code) pour supprimer les redondances.

5- retirer les colonnes ayant une correlation en valeur absolue > 70% avec les paramètres conservés.

In [5]:
```
#analyse redondance de parametres
correlation=100*df.corr()
#print(correlation.round(1))
```

In [6]:
```
#TO DO : recherche des parametres redondants et suppression
npara=correlation.shape[1]
seuil = 70
redondant=[]
asupprimer=[correlation.columns[j] for i in range(npara) for j in range(i + 1, npara) if abs(correlation.iloc[i
print(asupprimer)

""" Correction
npara=correlation.shape[1]
redondant=[]
seuil = 70
for i in range(npara-1):
    col = [c for c in range(i+1, npara) if abs correlation.iloc[i, c]
    #print(correlation.iloc[i, col])
    redondant.extend(col) #extend au lieu de append pour étendre une liste avec une liste
#suppression doublon
asupprimer=[]
for element in redondant:
    if element not in asupprimer:
        asupprimer.append(element)
print("numero des variables redondantes=",asupprimer)
print("noms des variables redondantes=",correlation.columns[asupprimer])
df=df.drop(correlation.columns[asupprimer],axis='columns') #equivalent axis=1
"""
```

```
['ifws', 'nbfws', 'lsws_ls', 'isws_ls', 'nbsws_ls', 'ifly_ls', 'iflo_ls', 'lflr_ls', 'lsws_hs', 'nbsws_hs', 'lfl
y_hs', 'nbfly_hs', 'lflr_hs', 'nbflr_hs', 'nbfws', 'lsws_ls', 'isws_ls', 'nbsws_ls', 'ifly_ls', 'iflo_ls', 'lflr
_ls', 'iflr_ls', 'nbflr_ls', 'lsws_hs', 'isws_hs', 'nbsws_hs', 'lfly_hs', 'ifly_hs', 'nbfly_hs', 'lflr_hs', 'ifl
r_hs', 'nbflr_hs', 'lsws_ls', 'isws_ls', 'nbsws_ls', 'ifly_ls', 'iflo_ls', 'lflr_ls', 'lsws_hs', 'nbsws_hs', 'lf
ly_hs', 'nbfly_hs', 'lflr_hs', 'nbflr_hs', 'isws_ls', 'nbsws_ls', 'ifly_ls', 'iflo_ls', 'lflr_ls', 'lsws_hs', 'n
bsws_hs', 'lfly_hs', 'nbfly_hs', 'lflr_hs', 'nbflr_hs', 'nbsws_ls', 'ifly_ls', 'iflo_ls', 'lflr_ls', 'iflr_ls',
'nbflr_ls', 'lsws_hs', 'isws_hs', 'lfly_hs', 'ifly_hs', 'nbfly_hs', 'lflr_hs', 'iflr_hs', 'nbflr_hs', 'ifly_ls',
'iflo_ls', 'lflr_ls', 'lsws_hs', 'nbsws_hs', 'lfly_hs', 'nbfly_hs', 'lflr_hs', 'nbflr_hs', 'iflo_ls', 'lflr_ls',
'iflr_ls', 'nbflr_ls', 'lsws_hs', 'isws_hs', 'nbsws_hs', 'lfly_hs', 'ifly_hs', 'nbfly_hs', 'lflr_hs', 'iflr_hs',
'nbflr_hs', 'hflo_ls', 'hflr_ls', 'hsws_hs', 'hfly_hs', 'hflr_hs', 'lflr_ls', 'iflr_ls', 'nbflr_ls', 'lsws_hs',
'isws_hs', 'nbsws_hs', 'lfly_hs', 'ifly_hs', 'nbfly_hs', 'lflr_hs', 'iflr_hs', 'nbflr_hs', 'hflr_ls', 'hfly_hs',
'hflr_hs', 'lsws_hs', 'nbsws_hs', 'lfly_hs', 'nbfly_hs', 'lflr_hs', 'nbflr_hs', 'nbflr_ls', 'isws_hs', 'ifly_hs'
, 'nbfly_hs', 'iflr_hs', 'hsws_hs', 'hfly_hs', 'hflr_hs', 'isws_hs', 'ifly_hs', 'nbfly_hs', 'iflr_hs', 'nbflr_hs
', 'nbsws_hs', 'lfly_hs', 'nbfly_hs', 'lflr_hs', 'nbflr_hs', 'ifly_hs', 'nbfly_hs', 'iflr_hs', 'nbflr_hs', 'hfly
_hs', 'hflr_hs', 'lfly_hs', 'nbfly_hs', 'lflr_hs', 'nbflr_hs', 'nbfly_hs', 'lflr_hs', 'nbflr_hs', 'nbfly_hs', 'i
flr_hs', 'hflr_hs', 'lflr_hs', 'nbflr_hs', 'nbflr_hs']
```

Out[6]:
```
' Correction\nnpara=correlation.shape[1]\nredondant=[]\nseuil = 70\nfor i in range(npara-1):\n    col = [c for
c in range(i+1, npara) if abs correlation.iloc[i, c]\n    #print(correlation.iloc[i, col])\n    redondant.exten
d(col) #extend au lieu de append pour étendre une liste avec une liste\n#suppression doublon\nasupprimer=[]\nfo
r element in redondant:\n    if element not in asupprimer:\n        asupprimer.append(element)\nprint("numero d
es variables redondantes=",asupprimer)\nprint("noms des variables redondantes=",correlation.columns[asupprimer]
)\ndf=df.drop(correlation.columns[asupprimer],axis=\'columns\') #equivalent axis=1        \n'
```

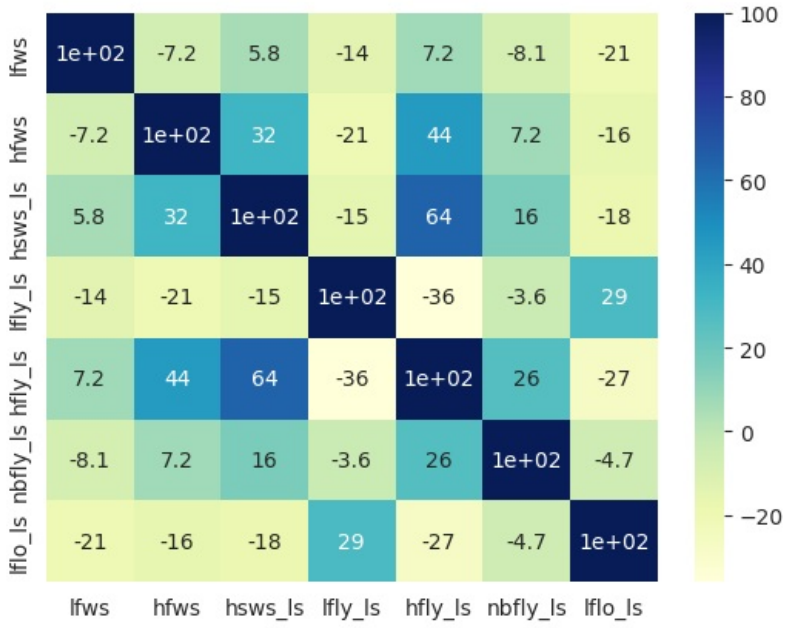
In [7]:
```
df_lite = df.copy()

for col in  df :
    if col in asupprimer :
        df_lite.drop(columns=col, inplace=True)

# TODO : recalculer des correlations entre les variables conservées et les afficher
correlation=100*df_lite.corr()
print(correlation.round(1))
seaborn.heatmap(correlation, cmap="YlGnBu", annot=True)
```

```
           lfws   hfws  hsws_ls  lfly_ls  hfly_ls  nbfly_ls  lflo_ls
lfws      100.0   -7.2      5.8    -14.3      7.2      -8.1    -20.7
hfws       -7.2  100.0     32.1    -20.9     44.3       7.2    -16.1
hsws_ls     5.8   32.1    100.0    -14.8     64.4      15.6    -18.4
lfly_ls   -14.3  -20.9    -14.8    100.0    -35.9      -3.6     29.0
hfly_ls     7.2   44.3     64.4    -35.9    100.0      26.2    -27.3
nbfly_ls   -8.1    7.2     15.6     -3.6     26.2     100.0     -4.7
lflo_ls   -20.7  -16.1    -18.4     29.0    -27.3      -4.7    100.0
```

Out[7]: <Axes: >



6- Créer un nouveau dataframe avec ces variables réduites et les labels initiaux. Afficher par paire de paramètres ce dataframe avec une coloration par label

In [8]:
```
#TO DO
df_lite.add(labels)
df_lite["class"] = labels
seaborn.pairplot(df_lite, hue="class")
```

Out[8]: <seaborn.axisgrid.PairGrid at 0x7f1989b72710>



Grâce à la visualisation par paire, on peut observer qu'il semble difficile de séparer les classes de manière linéaire. Il apparait également qu'il ne semble pas possible d'utiliser un algorithme comme celui des Kmeans car les classes ne semblent pas être séparée de manière euclidienne.

Partitionnement

En ajoutant au nouveau dataframe les labels et en faisant une visualisation par paire on remarqu'il semble que nos données ne soient pas globulaires, ni linéairement séparables. Une distance classique euclidienne et un kmeans ne permettra pas de séparer correctement nos données. Nous allons analyser si ces classes sont bien homogènes, n'existe-t-il pas des sous-classes qui faciliterait la prédiction par la suite ?

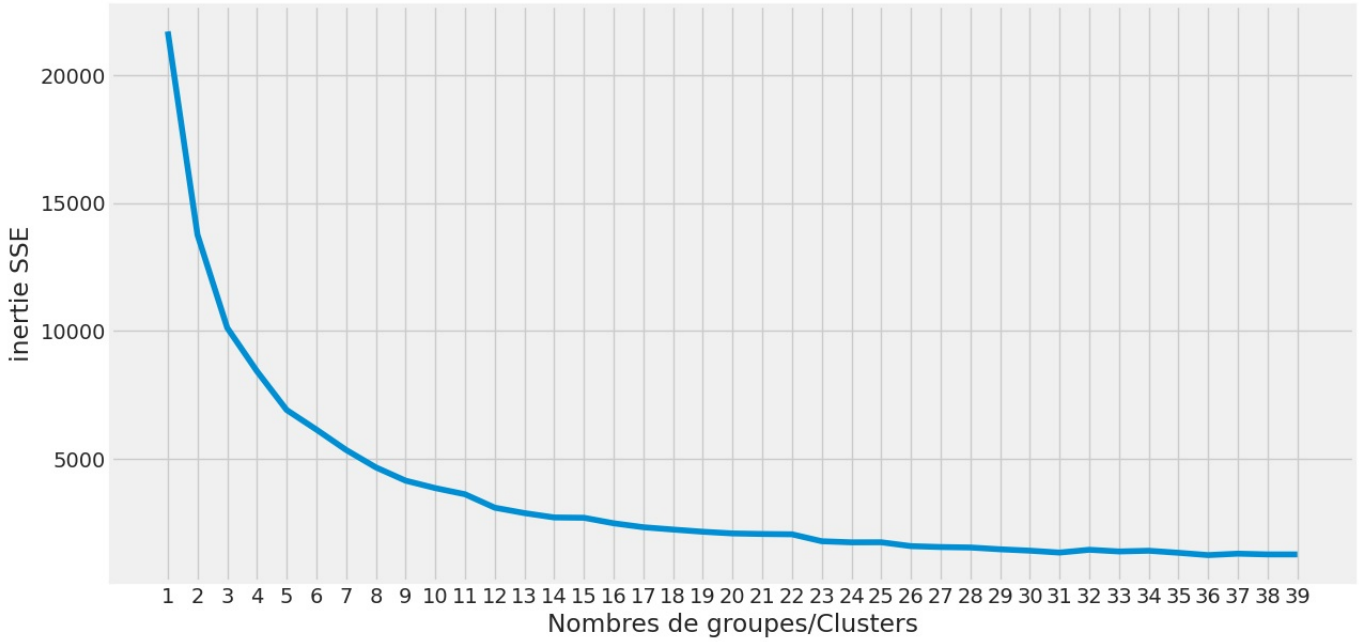
In [9]:
```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
from kneed import KneeLocator

# normalisation des données
scaler = StandardScaler() # instanciation
X = scaler.fit_transform(df) # normalisation et transformation

### etude si le nombre k de clusters est identique au K vrai=7
#creation d'une liste vide pour enregistrer les inerties sse fonction du nombre de
sse = []
kmax=40 # nombre de clusters max
for k in range(1, kmax): # boucle sur le nombre de clusters
    kmeans = KMeans(init="random",n_clusters=k,n_init=10,max_iter=300,random_state=10) # instanciation
    kmeans.fit(X) # application sur les données
    sse.append(kmeans.inertia_) # ajout de l'inertie dans la liste
```

```
# affichage de la courbe
plt.figure(figsize=(15,7))
plt.style.use("fivethirtyeight")
plt.plot(range(1, kmax), sse)
plt.xticks(range(1, kmax))
plt.xlabel("Nombres de groupes/Clusters")
plt.ylabel("inertie SSE")
plt.show()

# si kneed non installé commenter les lignes suivantes
#pour calculer automatiquement le nombre K
kl = KneeLocator(range(1, kmax), sse, curve="convex", direction="decreasing") # instanciation
kl.elbow # valeur de K
# K correspond à la valeur du coude de la courbe (le point d'inflexion)
```



Out[9]: 8

On remarque que malgré les 7 groupes de cellules, la partition serait en 8 groupes avec une inertie encore élevée.

7- Etudier les resultats d'un k-means avec k=8. Afficher la matrice de confusion.

In [16]:
```
# TO DO
# kmeans pour k=8
kmeans = KMeans(init="random",n_clusters=8,n_init=10,max_iter=300,random_state=10) # instanciation
kmeans.fit(X) # application sur les données

# affichage de la matrice de confusion entre les classes de départ et les celles du kmeans
from sklearn.metrics import confusion_matrix
from sklearn.preprocessing import LabelEncoder
LE = LabelEncoder()
labelsNum = LE.fit_transform(labels)

t = confusion_matrix(labelsNum, kmeans.labels_)
print(t)
```

```
[[75 24  0  0  0  0  0  1]
 [ 0  0  0 57  0  0  0 43]
 [ 1 43  0  0  0 31 25  0]
 [47  0  0  2 17  0  0 34]
 [ 0  0  0 21  0  0  0 79]
 [71  3  0  0  0  8  0 18]
 [ 0 84 14  0  0  1  1  0]
 [ 0  0  0  0  0  0  0  0]]
```

In [17]:
```
df_confusion = pd.crosstab(labels, kmeans.labels_, rownames=['Vrai'], colnames=['ParKmeans'], margins=True)
print(df_confusion)
```

```
ParKmeans                0    1   2   3   4   5   6    7  All
Vrai
chaetoceros_socialis    75   24   0   0   0   0   0    1  100
emiliana_huxleyi         0    0   0  57   0   0   0   43  100
lauderia_annulata        1   43   0   0   0  31  25    0  100
leptocylindrus_minimus  47    0   0   2  17   0   0   34  100
phaeocystis_globosa      0    0   0  21   0   0   0   79  100
skeletonema_costatum    71    3   0   0   0   8   0   18  100
thalassiosira_rotula     0   84  14   0   0   1   1    0  100
All                    194  154  14  80  17  40  26  175  700
```

8- Commenter cette table de confusion. Quelles sont les classes bien identifiées dans un groupe distinct ?

En regardant la matrice de confusion, on peut voir que :

- **chaetoceros_socialis :** La majoritée des observations ont été prédites comme appartenant à la classe 0. Il y a également 24 observations qui ont été prédites comme appartenant à la classe 1.
- **emiliana_huxleyi :** Les observations ont été divisées entre les classes 3 et 7.
- **lauderia_annulata :** Les observations sont très réparties entre les classes 1, 5, et 6.
- **leptocylindrus_minimus :** Les observations sont principalement réparties entre les classes 0, 4, et 7.
- **phaeocystis_globosa :** La majoritée des observations ont été prédites comme appartenant à la classe 7, mais il y a aussi des prédictions dans la classe 3.
- **skeletonema_costatum :** La majoritée des observations ont été prédites comme appartenant à la classe 0, mais il y a aussi des prédictions dans les classes 1, 5, et 7.
- **thalassiosira_rotula :** La majoritée des observations (84) ont été prédites comme appartenant à la classe 1, mais il y a aussi quelques prédictions dans les classes 2, 5, et 6.

On peut voir que les prédictions sont très dispersées, et que les classes ne sont pas bien séparées. De plus, on peut voir que les classes 0, 1 et 7 sont surreprésentées dans les prédictions, alors que les classes 2, 3, 4 et 6 le sont beaucoup moins. On peut donc en conclure que le partitionnement n'est pas satisfaisant. Malgré tout, on peut observer que les classes suivantes sont identifiées dans un groupe distinct :

- **chaetoceros_socialis :** 0
- **phaeocystis_globosa :** 7
- **skeletonema_costatum :** 0
- **thalassiosira_rotula :** 1

9- Recommencer avec un nombre k plus grand, k=14 voire k=21. Analyser

In [18]:
```
# kmeans pour k=14
kmeans = KMeans(init="random", n_clusters=14, n_init=10, max_iter=300, random_state=10)  # instanciation
kmeans.fit(X)  # application sur les données

# affichage de la matrice de confusion entre les classes de départ et les celles du kmeans
LE = LabelEncoder()
labelsNum = LE.fit_transform(labels)

t = confusion_matrix(labelsNum, kmeans.labels_)
df_confusion = pd.crosstab(labels, kmeans.labels_, rownames=['Vrai'], colnames=['ParKmeans'], margins=True)
print(df_confusion)
```

```
ParKmeans                0   1    2   3   4   5   6   7   8   9   10  11  12  \
Vrai
chaetoceros_socialis     0   0    0   0   0   0   0   0   0  46   44   0   3
emiliana_huxleyi         0   0   43  30   0   0   0  27   0   0    0   0   0
lauderia_annulata        0   0    0   0   0   0   0   0  25  13    0  25   2
leptocylindrus_minimus   0  11   36   0   0   0  16   2   0   0   35   0   0
phaeocystis_globosa      0   0    5   5  75   0   0  15   0   0    0   0   0
skeletonema_costatum     1   0   18   0   0   0   2   0   6  15   57   0   0
thalassiosira_rotula    16   0    0   0   0  14   0   0   0   5    0   1  64
All                     17  11  102  35  75  14  18  44  31  79  136  26  69

ParKmeans                13  All
Vrai
chaetoceros_socialis      7  100
emiliana_huxleyi          0  100
lauderia_annulata        35  100
leptocylindrus_minimus    0  100
phaeocystis_globosa       0  100
skeletonema_costatum      1  100
thalassiosira_rotula      0  100
All                      43  700
```

En utilisant un nombre de cluster plus grand (k=14), on peut voir que les classes 0, 1, 3, 4, 5, 6, 8, 11, et 12 du kmeans semblent être bien séparées, mais que les classes 2, 7, 9, 10, et 13 sont des mélanges de plusieurs labels, et ne sont donc pas correctement séparées. On peut tout de même noter une amélioration par rapport au kmeans avec k=8.

In [19]:
```
# kmeans pour k=21
kmeans = KMeans(init="random", n_clusters=21, n_init=10, max_iter=300, random_state=10)  # instanciation
kmeans.fit(X)  # application sur les données
```

```
# affichage de la matrice de confusion entre les classes de départ et les celles du kmeans
LE = LabelEncoder()
labelsNum = LE.fit_transform(labels)

t = confusion_matrix(labelsNum, kmeans.labels_)
df_confusion = pd.crosstab(labels, kmeans.labels_, rownames=['Vrai'], colnames=['ParKmeans'], margins=True)
print(df_confusion)
```

```
ParKmeans                0   1   2   3   4   5   6   7   8   9  ...  12  13  \
Vrai                                                            ...
chaetoceros_socialis     0  41   0   0   0   0   0   0   0   0  ...   0   0
emiliana_huxleyi         0   0   0   0   0   0   0   0   0   0  ...   0   0
lauderia_annulata        0  21  24   0  27   0   0   0   0  10  ...   0  17
leptocylindrus_minimus   1   0   0  15   0  32   0   0  25   0  ...   0   0
phaeocystis_globosa     34   0   0   0   0   0  37  14   0   0  ...   0   0
skeletonema_costatum     0   2   3   0   0   0   0   0   0   0  ...   0   0
thalassiosira_rotula     0   4   0   0  13   0   0   0   0   0  ...  14   1
All                     35  68  27  15  40  32  37  14  25  10  ...  14  18

ParKmeans                14  15  16  17  18  19  20  All
Vrai
chaetoceros_socialis     0   3  53   1   0   0   0  100
emiliana_huxleyi        43   0   0   0   0   0  30  100
lauderia_annulata        0   0   0   0   0   0   0  100
leptocylindrus_minimus  14   0   0   0  11   0   0  100
phaeocystis_globosa      2   0   0   0   0   7   5  100
skeletonema_costatum     0  56  17  22   0   0   0  100
thalassiosira_rotula     0   0   0   0   0   0   0  100
All                     59  59  70  23  11   7  35  700

[8 rows x 22 columns]
```

En utilisant un nombre de cluster encore plus grand (k=21), on peut voir que les classes 0, 2, 3, 5, 6, 7, 8, 9, 10, 11, 12, 13, 15, 17, 18, 19 et 20 sont plutôt bien séparées, mais que certaines classes sont des restes des mélanges de plusieurs labels. On peut de nouveau remarquer une amélioration par rapport a l'itération précédente, mais il parait difficilement envisageable de continuer à augmenter le nombre de cluster indéfiniment pour obtenir une séparation correcte des classes.

Classification par arbre de décision

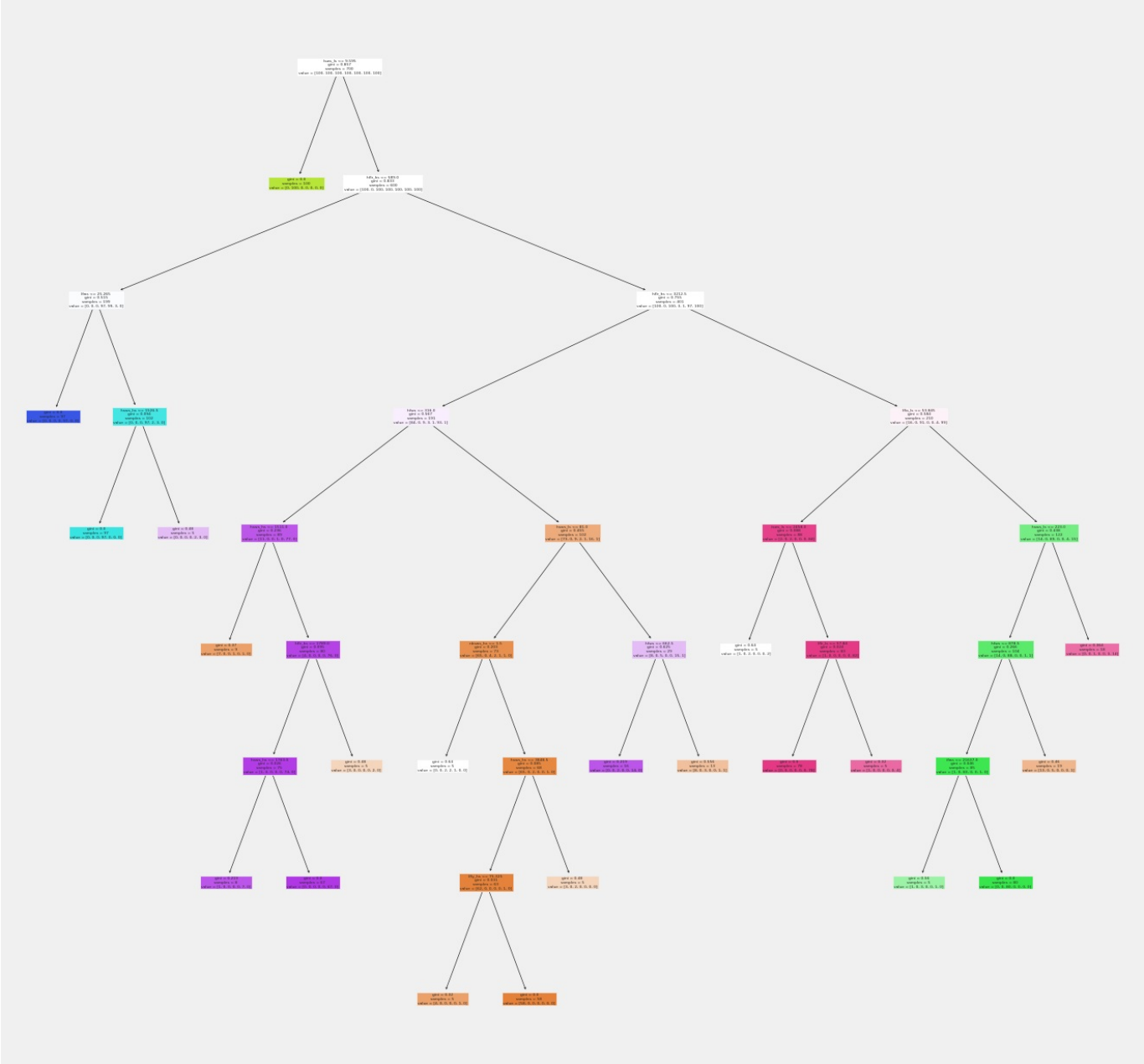
In [14]:
```
# instanciation de l'arbre
from sklearn.tree import DecisionTreeClassifier
arbreFirst = DecisionTreeClassifier(min_samples_split=20, min_samples_leaf=5)
# apprentissage des données
arbreFirst.fit(df, y=labels)
```

Out[14]:
```
DecisionTreeClassifier(min_samples_leaf=5, min_samples_split=20)
```

10- A quoi correspondent les paramètres fournis ?

- **min_samples_split :**
  - Il s'agit du nombre minimum d'échantillons requis pour diviser un nœud interne.
  - Si le nombre d'échantillons est inférieur à ce nombre, le noeud est considéré comme une feuille.
  - Si le nombre d'échantillons est supérieur ou égal à ce nombre, le noeud est considéré comme un noeud interne et pourra être divisé.
- **min_samples_leaf :**
  - Il s'agit du nombre minimum d'échantillons requis pour être à une feuille.
  - Si le nombre d'échantillons est inférieur à ce nombre, la division du noeud parent est annulée.
  - Si le nombre d'échantillons est supérieur ou égal à ce nombre, le noeud peut être une feuille.

In [15]:
```
# affichage de l'arbre
from sklearn import tree
plt.figure(figsize=(15,15))
tree.plot_tree(arbreFirst, feature_names=list(df), filled=True)
plt.show()
```



11- Que remarquez-vous sur le nombre de règles ? Pouvions-nous l'imaginez selon les résultats du k-means ?

On peut observer que cet arbre comporte un nombre conséquent de règles, ce que nous pouvions pressentir en observant les résultats du k-means. En effet, les classes étaient difficilement séparables, et il était donc difficile d'imaginer que l'arbre de décision serait 'simple'.

In [20]:
```
# visualisation des règles
from sklearn.tree import export_text
tree_rules = export_text(arbreFirst, feature_names=list(df))
print(tree_rules)
```

```
|--- lsws_ls <= 9.60
|   |--- class: emiliana_huxleyi
|--- lsws_ls >  9.60
|   |--- hflr_hs <= 589.00
|   |   |--- lfws <= 25.27
|   |   |   |--- class: phaeocystis_globosa
|   |   |--- lfws >  25.27
|   |   |   |--- hsws_hs <= 1526.50
|   |   |   |   |--- class: leptocylindrus_minimus
|   |   |   |--- hsws_hs >  1526.50
|   |   |   |   |--- class: skeletonema_costatum
|   |--- hflr_hs >  589.00
|   |   |--- hflr_hs <= 3212.50
|   |   |   |--- hfws <= 316.00
|   |   |   |   |--- hsws_hs <= 1511.00
|   |   |   |   |   |--- class: chaetoceros_socialis
|   |   |   |   |--- hsws_hs >  1511.00
|   |   |   |   |   |--- hflr_hs <= 1789.00
|   |   |   |   |   |   |--- hsws_hs <= 1763.00
|   |   |   |   |   |   |   |--- class: skeletonema_costatum
|   |   |   |   |   |   |--- hsws_hs >  1763.00
|   |   |   |   |   |   |   |--- class: skeletonema_costatum
|   |   |   |   |   |--- hflr_hs >  1789.00
|   |   |   |   |   |   |--- class: chaetoceros_socialis
|   |   |   |--- hfws >  316.00
|   |   |   |   |--- hsws_ls <= 81.00
|   |   |   |   |   |--- nbsws_hs <= 1.50
|   |   |   |   |   |   |--- class: lauderia_annulata
|   |   |   |   |   |--- nbsws_hs >  1.50
|   |   |   |   |   |   |--- hsws_hs <= 3848.50
|   |   |   |   |   |   |   |--- lfly_hs <= 75.33
|   |   |   |   |   |   |   |   |--- class: chaetoceros_socialis
|   |   |   |   |   |   |   |--- lfly_hs >  75.33
|   |   |   |   |   |   |   |   |--- class: chaetoceros_socialis
|   |   |   |   |   |   |--- hsws_hs >  3848.50
|   |   |   |   |   |   |   |--- class: chaetoceros_socialis
|   |   |   |   |--- hsws_ls >  81.00
|   |   |   |   |   |--- hfws <= 662.50
|   |   |   |   |   |   |--- class: skeletonema_costatum
|   |   |   |   |   |--- hfws >  662.50
|   |   |   |   |   |   |--- class: chaetoceros_socialis
|   |   |--- hflr_hs >  3212.50
|   |   |   |--- lflo_ls <= 53.84
|   |   |   |   |--- isws_ls <= 2458.00
|   |   |   |   |   |--- class: lauderia_annulata
|   |   |   |   |--- isws_ls >  2458.00
|   |   |   |   |   |--- lflr_ls <= 57.84
|   |   |   |   |   |   |--- class: thalassiosira_rotula
|   |   |   |   |   |--- lflr_ls >  57.84
|   |   |   |   |   |   |--- class: thalassiosira_rotula
|   |   |   |--- lflo_ls >  53.84
|   |   |   |   |--- hsws_ls <= 223.00
|   |   |   |   |   |--- hfws <= 878.50
|   |   |   |   |   |   |--- ifws <= 21637.00
|   |   |   |   |   |   |   |--- class: lauderia_annulata
|   |   |   |   |   |   |--- ifws >  21637.00
|   |   |   |   |   |   |   |--- class: lauderia_annulata
|   |   |   |   |   |--- hfws >  878.50
|   |   |   |   |   |   |--- class: chaetoceros_socialis
|   |   |   |   |--- hsws_ls >  223.00
|   |   |   |   |   |--- class: thalassiosira_rotula
```

Vous remarquez ici que l'espèce Emiliana est facilement identifiable à partir d'un unique paramètre. Ce que l'algorithme kmeans n'est pas capable d'isoler, car il considère l'ensemble des paramètres. A l'inverse des kmeans, certaines classes nécessitent un grand nombre de règles pour les isoler. L'utilisation des kmeans (et particulièrement spectral-kmeans) permet en cytométrie d'aider les experts biologistes à prétiqueter leurs données et visualiser seulement pour quelques données leurs images ou cytogrammes afin de les étiqueter.

In [23]:
```
prediction = arbreFirst.predict(X=df)
df_confusion = pd.crosstab(labels, prediction, rownames=['Vrai'], colnames=['ParArbre'], margins=True)
print(df_confusion)
```

```
ParArbre               chaetoceros_socialis  emiliana_huxleyi  \
Vrai
chaetoceros_socialis                     91                 0
emiliana_huxleyi                          0               100
lauderia_annulata                        11                 0
leptocylindrus_minimus                    2                 0
phaeocystis_globosa                       0                 0
skeletonema_costatum                      2                 0
thalassiosira_rotula                      2                 0
All                                     108               100

ParArbre               lauderia_annulata  leptocylindrus_minimus  \
Vrai
chaetoceros_socialis                   2                       0
emiliana_huxleyi                       0                       0
lauderia_annulata                     82                       0
leptocylindrus_minimus                 0                      94
phaeocystis_globosa                    0                       0
skeletonema_costatum                   3                       0
thalassiosira_rotula                   0                       0
All                                   87                      94

ParArbre               phaeocystis_globosa  skeletonema_costatum  \
Vrai
chaetoceros_socialis                     0                     5
emiliana_huxleyi                         0                     0
lauderia_annulata                        0                     4
leptocylindrus_minimus                   4                     0
phaeocystis_globosa                    100                     0
skeletonema_costatum                     0                    93
thalassiosira_rotula                     1                     1
All                                    105                   103

ParArbre               thalassiosira_rotula  All
Vrai
chaetoceros_socialis                      2  100
emiliana_huxleyi                          0  100
lauderia_annulata                         3  100
leptocylindrus_minimus                    0  100
phaeocystis_globosa                       0  100
skeletonema_costatum                      2  100
thalassiosira_rotula                     96  100
All                                     103  700
```

12- Commentez le role de chaque

`prediction = arbreFirst.predict(X=df)` permet de prédire les labels des observations en utilisant l'arbre de décision que nous avons créé précédemment. `df_confusion = pd.crosstab(labels, prediction, rownames=['Vrai'], colnames=['ParArbre'], margins=True)` permet de créer une matrice de confusion entre les labels de départ et les labels prédits par l'arbre de décision. `print(df_confusion)` permet d'afficher la matrice de confusion. On peut observer que grâce à l'arbre de décision, la classification est bien plus réussie qu'avec l'algorithme des kmeans. Les classes sont plutôt bien séparer, avec peu d'erreur.

13- (Bonus) Faire deux bases, une pour l'apprentissage et une pour le test, et étudier l'impact des paramètres de l'arbre